Llama 模型

第 4 章（下）

第 6 课时 本课时讲解 4.3–4.6

Llama 模型 · 模型结构 · 键值缓存 · 长文本继续预训练

著者
邓杭　杨耀东

教材
《大语言模型：训练、精调与对齐》

出版
清华大学出版社



本章目录

第 6 课时

4.1 Llama 模型
开源里程碑与预训练规模 第 3 页

4.2 Llama 模型结构
整体结构、Llama 块与四个关键模块 第 14 页

4.3 键值缓存
生成分析、实现与显存 第 44 页

4.4 长文本继续预训练
扩展方案与位置编码改造 第 58 页

4.5 小结
结构要点与后续衔接 第 77 页

4.6 附录：稀疏混合专家系统
Top-K 门控与负载均衡 第 80 页

本课时内容 4.3 键值缓存、4.4 长文本继续预训练、4.5 小结、4.6 附录：稀疏混合专家系统（其余部分见第 5 课时）

4.3 键值缓存

自回归生成每次只新增一个词元，却反复重算整段历史。KV-Cache 用空间换时间：把已经算过的键和值留下来。

本节内容

4.3.1 自回归模型生成分析

4.3.2 KV-Cache 的实现

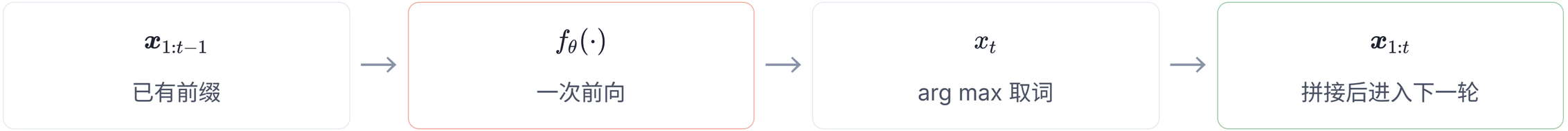
4.3.3 KV-Cache 显存消耗

逐词元生成的计算过程

解码器模型推理时，需要输入词元序列并逐个词元生成。长度为 $t-1$ 的输入 $\boldsymbol{x}_{1:t-1}$ 前向计算得到 x_t，新词元再用于下一轮得到 x_{t+1}。

$$f_\theta(\boldsymbol{x}_{1:t-1}) = \arg\max_{v \in \mathcal{V}} p_\theta(x_v \mid \boldsymbol{x}_{1:t-1}) \tag{4-44}$$

贪婪解码：在词表 $\mathcal{V}$ 中取概率最大的词元。$p_\theta(x_v \mid \boldsymbol{x}_{1:t-1})$ 是模型对下一个词元 v 的预测分布。



生成 T 个词元就要做 T 次前向，而每次的输入只比上一次多一个词元——这正是重复计算的来源。

依据 《大语言模型：训练、精调与对齐》第 4.3.1 节（式 (4-44)）

无缓存时的重复计算

输入 $\boldsymbol{x}_{1:t}$ 经嵌入层得到 $\boldsymbol{X}_{1:t}$，再投影并计算注意力（$\mathtt{Attn}(\cdot)$ 为缩放点积注意力）：

$$\boldsymbol{Q}_{1:t} = \boldsymbol{X}_{1:t}\boldsymbol{W}_q \tag{4-45}$$

$$\boldsymbol{K}_{1:t} = \boldsymbol{X}_{1:t}\boldsymbol{W}_k \tag{4-46}$$

$$\boldsymbol{V}_{1:t} = \boldsymbol{X}_{1:t}\boldsymbol{W}_v \tag{4-47}$$

$$\boldsymbol{Z}_{1:t} = \mathtt{Attn}(\boldsymbol{Q}_{1:t}, \boldsymbol{K}_{1:t}, \boldsymbol{V}_{1:t}) \tag{4-48}$$

$\boldsymbol{Z}_{1:t} \in \mathbb{R}^{t\times d}$：一次并行得到 t 个位置的特征，但自回归生成只需要最后一个位置。

$$\boldsymbol{Q}_{1:t+1} = \boldsymbol{X}_{1:t+1}\boldsymbol{W}_q \tag{4-49}$$

$$\boldsymbol{K}_{1:t+1} = \boldsymbol{X}_{1:t+1}\boldsymbol{W}_k \tag{4-50}$$

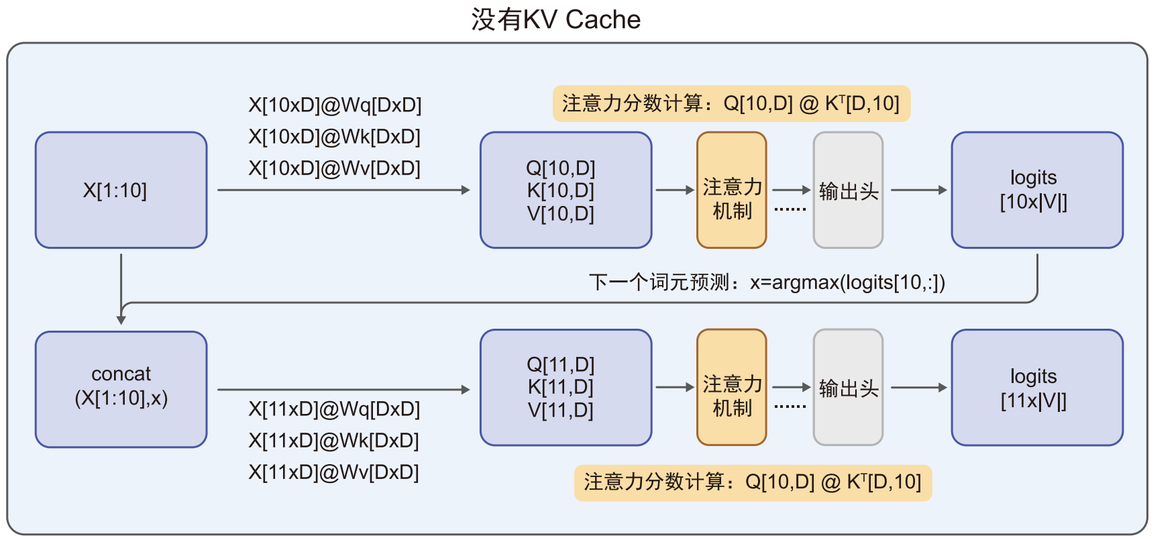
$$\boldsymbol{V}_{1:t+1} = \boldsymbol{X}_{1:t+1}\boldsymbol{W}_v \tag{4-51}$$

$$\boldsymbol{Z}_{1:t+1} = \mathtt{Attn}(\boldsymbol{Q}_{1:t+1}, \boldsymbol{K}_{1:t+1}, \boldsymbol{V}_{1:t+1}) \tag{4-52}$$

$\boldsymbol{x}_{:t+1} \leftarrow (\boldsymbol{x}_{:t}, x_{t+1})$：新词元拼到末尾，但 $\boldsymbol{X}_{1:t}\boldsymbol{W}_k$、$\boldsymbol{X}_{1:t}\boldsymbol{W}_v$ 又被算了一遍。

依据 《大语言模型：训练、精调与对齐》第 4.3.1 节（式 (4-45)–(4-48)）

重复计算的位置



- 处理 $\boldsymbol{x}_{1:t}$ 时并行算了 t 个位置的预测，实际只要最后一个位置。
- $t+1$ 时刻重复计算了 t 时刻已得到的 $\boldsymbol{K}_{1:t}$、$\boldsymbol{V}_{1:t}$——这两项可以缓存。
- $t+1$ 时刻还算了多个查询向量的注意力输出，实际只要第 $t+1$ 个位置。

图 4.10　无 KV-Cache 时，推理时注意力存在大量的重复计算

图中的 $\boldsymbol{X}[10\times D]@\boldsymbol{W}_k[D\times D]$ 与 $\boldsymbol{X}[11\times D]@\boldsymbol{W}_k[D\times D]$ 只差最后一行，前 10 行完全相同。

图 《大语言模型：训练、精调与对齐》图 4.10

键值缓存的计算方式

首次计算时把各注意力模块生成的 $\boldsymbol{K}$、$\boldsymbol{V}$ 存为 KV-Cache；之后每步只需处理最新词元 x_{t+1}。

$$\boldsymbol{Q}_{t+1} = \boldsymbol{X}_{t+1}\boldsymbol{W}_q \tag{4-53}$$

$$\boldsymbol{K}_{t+1} = \boldsymbol{X}_{t+1}\boldsymbol{W}_k \tag{4-54}$$

$$\boldsymbol{V}_{t+1} = \boldsymbol{X}_{t+1}\boldsymbol{W}_v \tag{4-55}$$

$$\boldsymbol{K}_{1:t+1} \leftarrow \texttt{Concat}(\boldsymbol{K}_{1:t}, \boldsymbol{K}_{t+1}) \tag{4-56}$$

$$\boldsymbol{V}_{1:t+1} \leftarrow \texttt{Concat}(\boldsymbol{V}_{1:t}, \boldsymbol{V}_{t+1}) \tag{4-57}$$

$$\boldsymbol{Z}_{t+1} = \texttt{Attn}(\boldsymbol{Q}_{t+1}, \boldsymbol{K}_{1:t+1}, \boldsymbol{V}_{1:t+1}) \tag{4-58}$$

$\boldsymbol{Q}_{t+1}, \boldsymbol{K}_{t+1}, \boldsymbol{V}_{t+1} \in \mathbb{R}^{1\times d}$。Concat 沿序列维度拼接缓存与当前步；$\boldsymbol{Z}_{t+1}$ 只有一个位置。

- 只需单个查询向量 $\boldsymbol{Q}_{t+1}$ 与完整键值序列 $\boldsymbol{K}_{1:t+1}$、$\boldsymbol{V}_{1:t+1}$ 计算注意力特征。
- 历史序列的投影变换不再重算，注意力输出也不再为 t 个位置重复计算。

依据 《大语言模型：训练、精调与对齐》第 4.3.1 节（式 (4-49)–(4-52)）

预填充与解码两个阶段

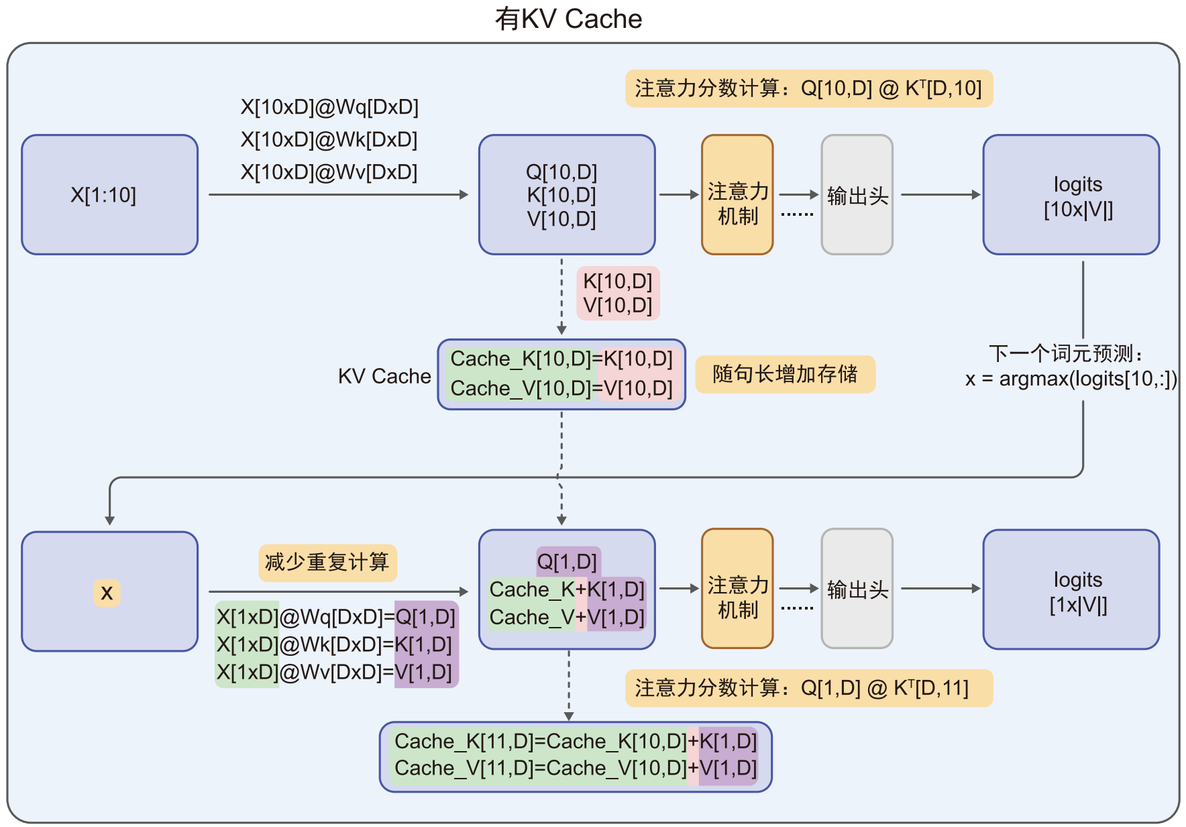


图 4.11　有 KV-Cache 时，生成过程可分为预填充和解码阶段

预填充（Prefill）

首次处理完整输入序列，在每层注意力中计算并缓存 $\boldsymbol{K}$、$\boldsymbol{V}$。

解码（Decoding）

每步只处理 1 个新词元，与缓存键值对一起算注意力，输出下一个词元。

缓存随句长增加而增长；换来的是解码阶段不再重复历史投影。

键值缓存的复杂度

KV-Cache 是注意力类模型的通用推理加速方法，适用于所有含解码器的 Transformer，并非 Llama 首创。

不使用 KV-Cache

每步都重算长度 t 的序列投影 $\boldsymbol{X}_{:t}\boldsymbol{W}$。

每步都要算 t 个位置的注意力输出。

时间复杂度随序列长度增加而显著增加。

使用 KV-Cache

解码阶段只算新词元的投影 $\boldsymbol{X}_{t+1}\boldsymbol{W}$。

只算单个查询向量 $\boldsymbol{Q}_{t+1}$ 的注意力输出。

计算量大幅减少，代价是额外的 KV-Cache 存储。

> **权衡** 用空间换时间。生成长序列时，这一权衡通常能大幅提升推理效率；而缓存本身的显存开销，就是 4.3.3 要解决的问题。

生成循环

预填充阶段需要完整序列 $\boldsymbol{x}_{1:t}$ 预测下一个词元；解码阶段有缓存，每轮输入 x_t 只有 1 个词元。

```
x = torch.randint(low=1, high=vocab_size, size=(1, seq_len))
mask = torch.triu(torch.ones(1, 1,
                             seq_len+max_new_tokens,
                             seq_len+max_new_tokens,
                             dtype=torch.bool))[:, :, 1:, :-1]
pos = torch.arange(seq_len)

# 带 KV-Cache 的模型
model = DecodingModelWithKVCache(dim, heads, vocab_size)
for i in range(max_new_tokens):
    logits = model(x, mask[:, :, pos, pos])
    next_tokens = torch.argmax(logits, dim=-1)

    # x = torch.cat((x, next_tokens[:,[-1]]), dim = -1)
    x = next_tokens[:, [-1]]  # 有 KV-Cache 更新情况

    pos = torch.tensor([seq_len+i])
    if next_tokens[0, -1] == EOS_token_ID:
        break
```

与无缓存版本的关键差别在注释一行：不再把新词元拼回输入，而是只把最新词元送入模型，历史由缓存承担。

注意力层的初始化与投影

```
class MultiHeadAttentionWithKVCache(nn.Module):
    def __init__(self, dim, heads):
        super(MultiHeadAttentionWithKVCache, self).__init__()
        self.heads = heads
        self.dim = dim
        self.WQ = nn.Linear(dim, dim)
        self.WK = nn.Linear(dim, dim)
        self.WV = nn.Linear(dim, dim)
        self.WO = nn.Linear(dim, dim)
        self.cache_K = None
        self.cache_V = None

    def forward(self, X, mask):
        B, L, D = X.size()
        head_dim = self.dim // self.heads  # 单头词向量维度
        Q, K, V = self.WQ(X), self.WK(X), self.WV(X)
        Q = Q.reshape(B, L, self.heads, head_dim).transpose(1, 2)
        K = K.reshape(B, L, self.heads, head_dim).transpose(1, 2)
```

- 四个线性层都不带偏置，与 4.2.1 的取舍一致；cache_K、cache_V 初始为 None。
- head_dim = dim // heads 即单头维度 d'；Q, K, V 被 reshape 成 (B, heads, L, head_dim)。
- 投影只作用于本步输入 X：解码阶段 $L = 1$，投影开销与序列长度无关。

缓存拼接与注意力计算

```
if self.cache_K is None:
    self.cache_K = K
    self.cache_V = V
else:
    self.cache_K = torch.cat((self.cache_K, K), dim=2)
    self.cache_V = torch.cat((self.cache_V, V), dim=2)

Z = ScaledDotProductAttention(Q,
                              self.cache_K, self.cache_V,
                              mask)
Z = Z.transpose(1, 2).reshape(B, L, self.dim)
O = self.WO(Z)
return O
```

- 首次调用对应预填充：直接把本步的 K、V 赋给缓存。
- 之后沿序列维 `dim=2` 拼接：缓存始终是"历史 + 当前步"的完整 $\boldsymbol{K}$、$\boldsymbol{V}$。
- 注意力的查询只来自本步输入 Q（解码时 $L=1$），键与值来自完整缓存。

依据 《大语言模型：训练、精调与对齐》第 4.3.2 节（代码 5）

输出投影

```
class DecodingModelWithKVCache(nn.Module):
    def __init__(self, dim, heads, vocab_size):
        super(DecodingModelWithKVCache, self).__init__()
        self.embd = nn.Embedding(vocab_size, dim)  # 忽略位置编码
        self.decoder = MultiHeadAttentionWithKVCache(dim, heads)  # 简化解码器
        self.norm = nn.LayerNorm(dim)
        self.lm_heads = nn.Linear(dim, vocab_size)

    def forward(self, x, mask):
        X = self.embd(x)
        X = self.decoder(X, mask)
        H = self.norm(X)
        logits = self.lm_heads(H)
        return logits
```

- 缓存管理加在注意力模块内部：预填充存储，解码时合并当前步的单个 KV 再计算注意力。
- 示例模型把解码器简化为单层注意力，位置编码由调用方以 pos 索引掩码的方式处理。

键值缓存的大小

KV-Cache 用空间换时间。解码阶段，缓存要同时保存键与值，其存储量为：

$$\Psi = l \times t \times d' \times h' \times k_{\mathrm{b}} \times 2 \tag{4-59}$$

l 为注意力层数，t 为序列长度，$d' = d/h$ 为每个注意力头的维度，h' 为需要缓存的键值头数（MHA 下 $h' = h$，GQA 下 $h' = g$），k_{b} 为存储数值类型的位数，末尾的 2 表示键与值各存一份。

$\propto t$

随序列长度线性增长

$\propto l$

每层各存一份

$\times 2$

键与值

长序列下缓存迅速膨胀，容易导致显存不足——这正是下一组优化方向的出发点。

减小缓存的五个方向

$\Psi = l \times t \times d' \times h' \times k_{\mathrm{b}} \times 2$ 的每个因子，都对应一条优化路径。

缩短 t

减少序列长度

窗口注意力限制计算范围；或用稀疏注意力减少键值对数量。

降低 l

层间缓存共享

相邻层共用 KV-Cache，降低层数因子的影响。

减少 h'

优化注意力头

用 GQA 或 MQA 减少需要存储的键值头数量。

压缩 d'

降低表示维度

学习低维潜在表示（$\tilde{d} \ll d$，$h = 1$）作为缓存，如 MLA。

第五条是压缩存储精度（k_{b}）：对 KV-Cache 量化。此外，分页注意力借鉴操作系统的分页机制提高存储利用率；部署时还可按预填充（计算密集）与解码（访存密集）分离算力。

依据 《大语言模型：训练、精调与对齐》第 4.3.3 节

GQA 的缓存大小

$$\Psi_t = l \times t \times d' \times h' \times k_{\mathrm{b}} \times 2 \tag{4-60}$$

h' 为分组后的键值头数量。MHA 需缓存 h 个头，GQA 只需 g 个，缓存降为原来的 g/h。

解码阶段：处理第 i 层注意力时，先算注意力分数，再把 h' 个键值头的数据在各自的查询组内复制，以匹配组内查询头数量。

存 h' 个 KV 头

缓存只保留组级状态

→

组内复制

匹配该组查询头数

→

算注意力分数

组内 h' 个查询头

→

仍只存 h' 个

计算后不扩大缓存

- 复制只发生在计算注意力分数之前，是临时的；计算完成后，KV-Cache 仍然仅存储 h' 个头的状态。
- 缓存变小意味着同样的显存可以容纳更大批次，批量解码的吞吐随之提升。

依据 《大语言模型：训练、精调与对齐》第 4.3.3 节（式 (4-60)）

4.4

长文本继续预训练

上下文从 8192 扩到 128k，靠的不是加长输入，而是改位置编码并让模型重新适应。先看衰减为何失效，再看 PI 与 NTK-RoPE 如何修复。

本节内容

4.4.1 Llama-3 长文本训练方案

4.4.2 衰减特性分析*

4.4.3 旋转位置编码分析

4.4.4 位置编码内插

4.4.5 NTK-RoPE

4.4.6 长文本能力测评

长文本继续预训练

继续预训练（Continue Pre-Training，CPT）是在常规预训练之外，让模型掌握特定领域知识或能力的手段。

语言

特定语种文本

增强模型在该语言上的能力。

领域

代码与数学

提升相应领域的理解与生成能力。

垂直

专业知识

注入行业内容，掌握专业表述。

- 长上下文处理是理解复杂任务的基础：长篇小说、海量网页汇总、复杂文档查询、大型代码工程，都要求模型在超长上下文中辨识并利用关键信息。
- 受训练资源限制，预训练会预设最大上下文长度，输入文本按此裁剪，模型也只适应这一长度的数据分布。
- 扩展有效上下文长度的可行途径，是通过预训练或微调继续训练。

上下文长度的分阶段扩展

Llama-3 的上下文可达 128 000：在已完成预训练（基础长度 8192）的模型上分阶段抬高目标长度；DeepSeek-V3 则用更激进的 4k → 32k → 128k 两阶段。



- 每一阶段的目标长度翻倍，模型逐级适应新的位置分布，而不是一步跳到 128k。
- 为什么要改位置编码：原始 RoPE 在 16k–64k 的相对距离上，点积分数并未如预期有效衰减。办法是缩放底数 $b = 10\,000$（4.4.5）；位置编码一变，就必须继续训练让模型适应新的分布。

另一条路线 · DeepSeek-V3

4k → 32k → 128k：跨度更大的两阶段扩展

先用 32k 继续预训练约 1000 步，再用 128k 继续预训练约 1000 步；**位置编码的超参随阶段调整**，训练量远小于重新预训练。

依据 《大语言模型：训练、精调与对齐》第 4.4.1 节

长距离下的注意力分数

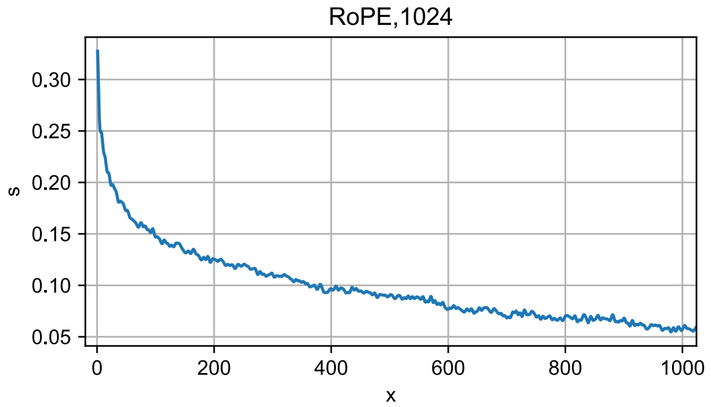


图 4.12(a)　上下文 1024：点积随相对距离稳步下降

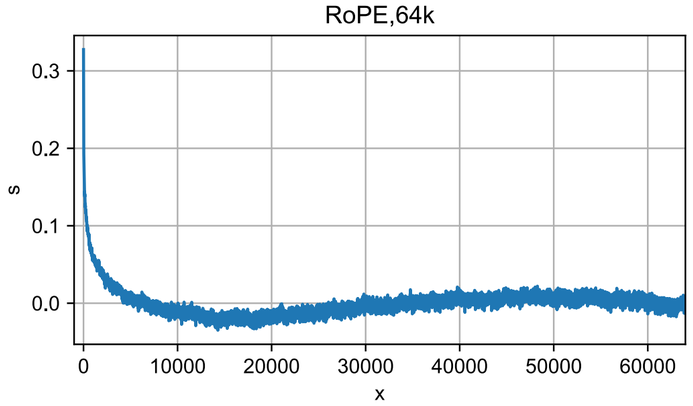


图 4.12(b)　上下文 64k：16k 之后点积停在 0 附近，不再衰减

$$\boldsymbol{q}_m \boldsymbol{R}_{m-n} \boldsymbol{k}_n^T$$

位置 m 与 n 的注意力分数；设 $\boldsymbol{q}_m, \boldsymbol{k}_n$ 为标准正态随机向量。

- 在 16k–64k 的相对距离上，点积分数没有按预期衰减。
- 距离越远、相关性本应越低；衰减失效意味着模型难以有效建模远距离依赖。
- 修法不是加长输入，而是改位置编码的表示方式，让长距离上稳定衰减、少震荡。

注意力分数的余弦形式

沿用 4.2.5 的 RoPE 形式，为分析随相对距离 $t = m - n$ 的衰减，取 $\boldsymbol{q}_m = \boldsymbol{k}_n = \mathbf{1} \in \mathbb{R}^d$（各元素独立同分布、均值 0、方差 1 的简化）：

$$f_t = \boldsymbol{q}_m \boldsymbol{R}_{m-n} \boldsymbol{k}_n^T \tag{4-61}$$

$$f_t = \mathbf{1}\, \boldsymbol{R}_t\, \mathbf{1}^T \tag{4-62}$$

$$= \sum_{i=1}^{d/2} \sin(m\theta_i)\sin(n\theta_i) + \cos(m\theta_i)\cos(n\theta_i) \tag{4-63}$$

$$= \sum_{i=1}^{d/2} \cos\left((m-n)\theta_i\right) \tag{4-64}$$

$$= \sum_{i=1}^{d/2} \cos\left(t\theta_i\right) \tag{4-65}$$

式 (4-63)：每组二维旋转贡献 $\sin\cdot\sin + \cos\cdot\cos$；式 (4-64)：用和差角公式合并为 $\cos((m-n)\theta_i)$，相对距离显式出现。

固定 $n = 0$ 时的衰减

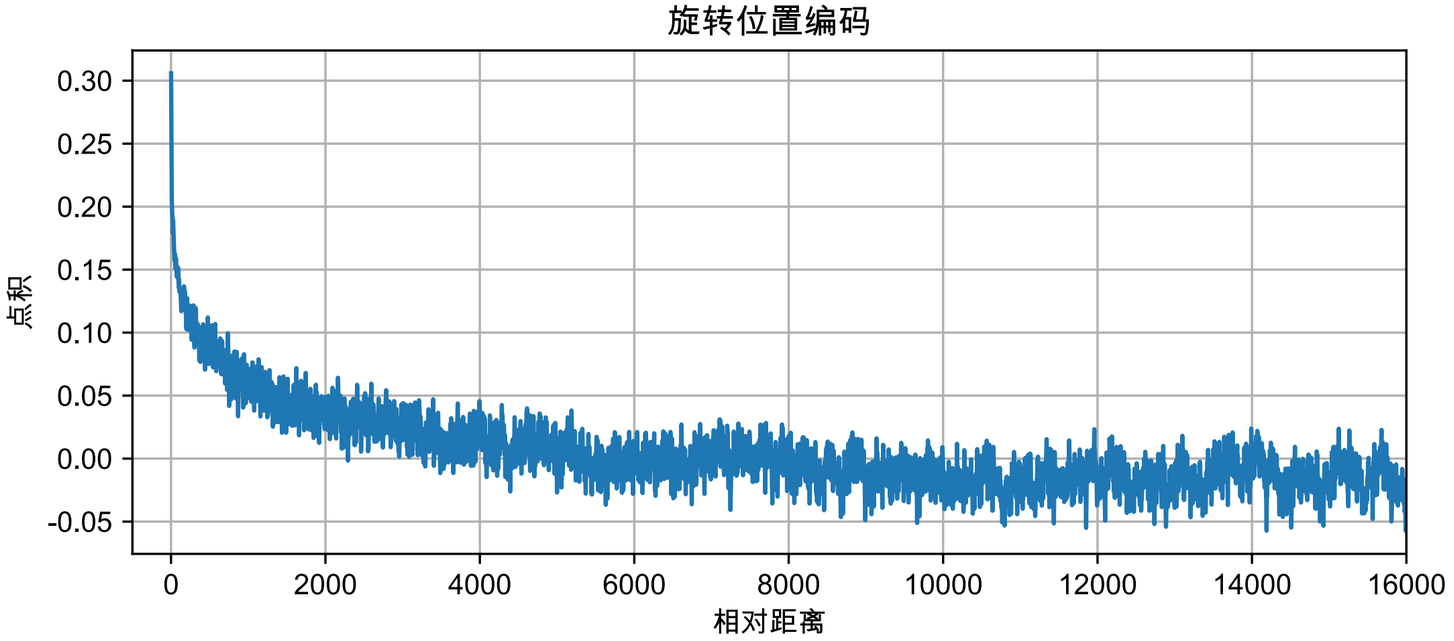


图 4.13　RoPE 位置编码点积衰减曲线图

固定键向量位置 $n = 0$，则 $t = m - 0 = m$，f_t 记为 f_m。

$$f_m = \sum_{i=1}^{d/2} \cos(m\theta_i) \quad (4\text{-}66)$$

$\theta_i = b^{-2(i-1)/d}$，共 $d/2$ 个不同频率的余弦叠加。

- 短相对距离内 f_m 明显衰减。
- 衰减不严格单调，在零附近震荡；f_m 可以为负。
- 包络的量级约为 $1/m$，下一节的证明给出 $|f_m| \leqslant 2/m$。

图 《大语言模型：训练、精调与对齐》图 4.13；式 (4-66)

衰减的证明

角度 θ_i 分布在 $[\alpha, \beta]$ 上，$\beta = \theta_1 > \theta_2 > \cdots > \theta_{d/2} = \alpha$。当 $d \to \infty$、角度间隔足够小且均匀时，离散求和可近似为积分：

$$f_m \approx \int_{\alpha}^{\beta} \cos(mx)\,\mathrm{d}x \tag{4-67}$$

$$|f_m| \approx \left| \int_{\alpha}^{\beta} \cos(mx)\,\mathrm{d}x \right| = \left| \left[\frac{1}{m} \sin(mx) \right]_{\alpha}^{\beta} \right| = \frac{1}{m} \left| \sin(m\beta) - \sin(m\alpha) \right| \tag{4-68}$$

式 (4-68)：$\cos(mx)$ 的原函数为 $\frac{1}{m}\sin(mx)$，代入上下限作差。

$$|f_m| \leqslant \frac{2}{m} \tag{4-69}$$

正弦值域为 $[-1, 1]$，故 $|\sin(m\beta) - \sin(m\alpha)| \leqslant 2$；当 $m \to \infty$ 时 $2/m \to 0$，因此 $|f_m| \to 0$。证毕。

- 结论：f_t 的绝对值随相对距离增大而震荡衰减并趋近 0，注意力天然偏向较近的上下文。
- 但衰减的非单调性（图 4.13）会给学习长距离依赖带来困难，增加长文本建模的难度。

依据 《大语言模型：训练、精调与对齐》第 4.4.2 节（式 (4-67)–(4-69)）

频率与周期的关系

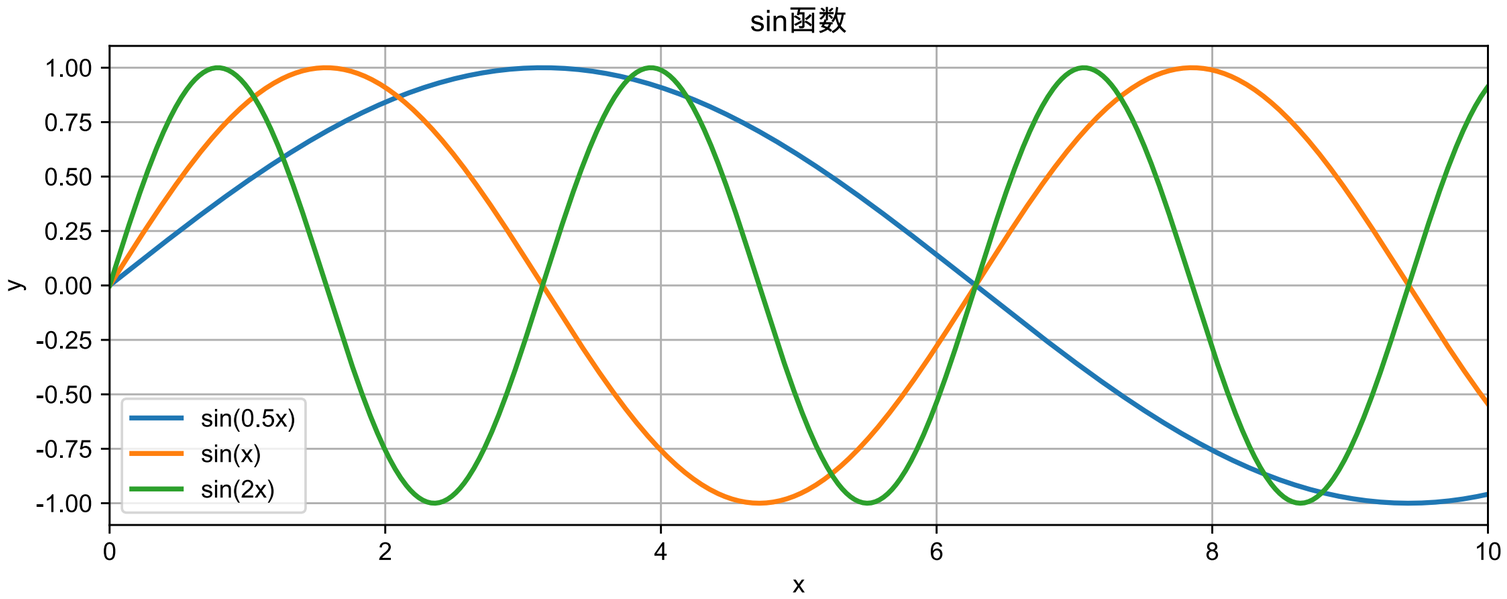


图 4.14　正弦函数周期与频率的关系：周期越短，频率越高

- 绝对位置编码与 RoPE 的元素都由周期函数算得。
- $\sin(\omega x)$ 的周期为 $2\pi/|\omega|$：ω 增大，周期变短、频率升高。
- 图 4.14 对比 $\omega = \frac{1}{2}, 1, 2$ 三种情形：波形从左到右依次变密。

这就是理解“高频组 / 低频组”的起点：RoPE 的不同维度对应不同的 ω。

各维度的频率分布

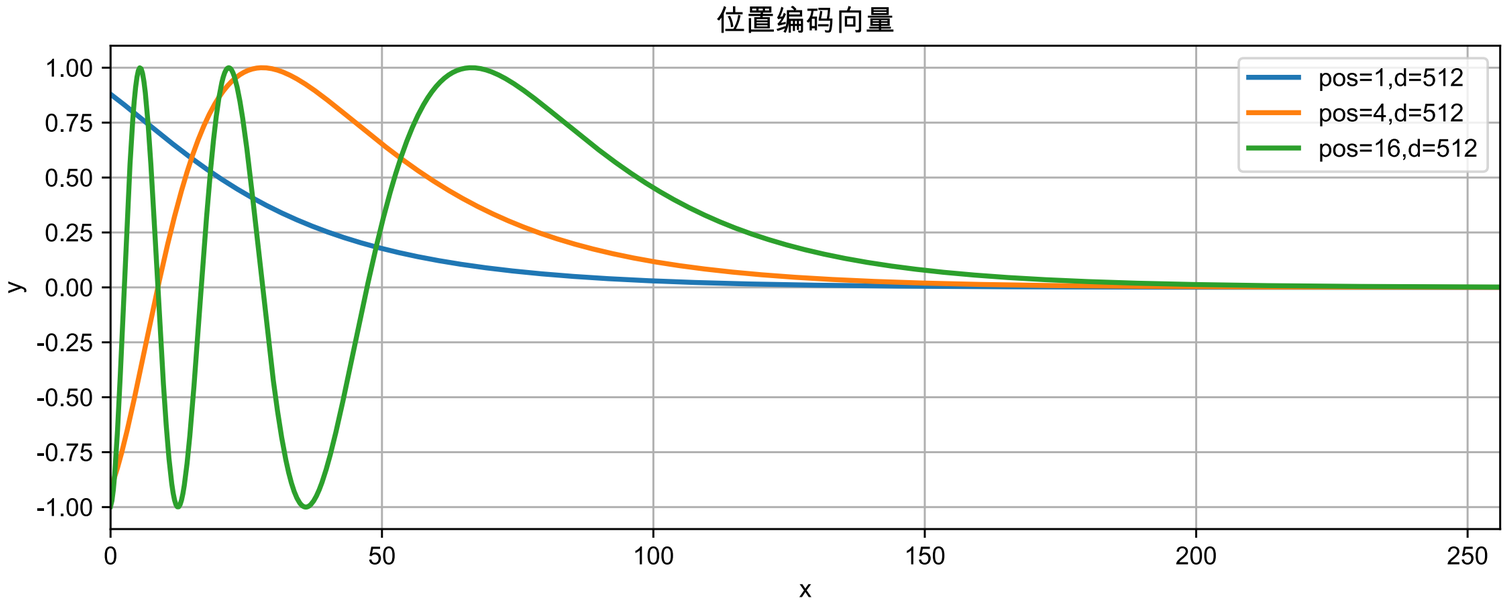


图 4.15　位置编码向量随维度的变化图

- RoPE 中旋转角度 $\theta_i = b^{-2(i-1)/d}$ 随维度 i 增加而变小。
- 向量左侧为低维信号，对应高频信息；右侧为高维信号，对应低频信息。
- 图 4.15 画出位置 $\{1, 4, 16\}$ 的编码向量：低维变换更快，高维变化更缓。

> **直觉**　高频维负责区分相邻位置的细微差别，低频维负责刻画长程的位置关系。

分频段的衰减特性

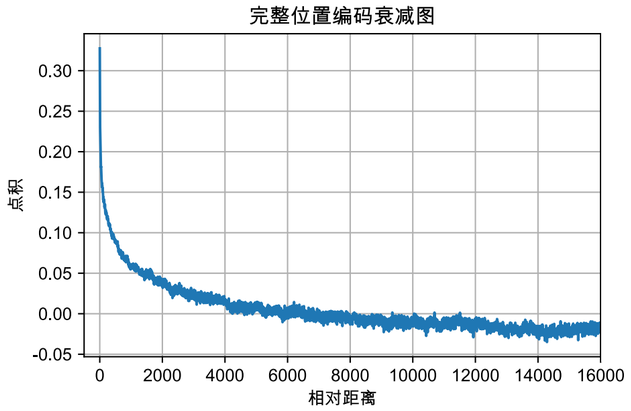


图 4.16(a)　全部维度合起来：连续下降后在 0 附近小幅振荡

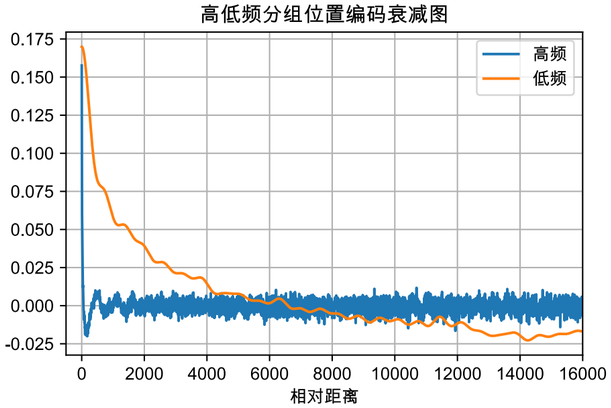


图 4.16(b)　按维度分成两半：高频组迅速衰减并振荡，低频组持续缓慢下降

把经 RoPE 变换的向量按维度分成两半，分别作为低频组与高频组，各自计算点积。

- 高频组：短距离上衰减更好；长距离时点积收敛到一个区间，并出现周期性振荡。
- 低频组：长距离下能保持更稳定的衰减。
- 这解释了两者互补：短程区分靠高频，长程建模靠低频。

位置内插（PI）

位置编码内插（Position Interpolation，PI）用于扩展上下文窗口。直接外推——让模型处理超出预训练长度的输入——会让位置编码值越出模型见过的数值分布，注意力分布因此混乱或过度集中于局部片段。

$$R(m') = R(m/s) \tag{4-70}$$

$s > 1$ 为内插系数，m 为原始位置索引，m' 为压缩后的新位置。PI 的核心思想是对位置索引压缩，把超长位置映射回预训练时见过的范围。

例 1

$s = 2$ 时，原始位置 $m = 2$ 映射到新位置 $m' = 1$。

例 2

$m = 17$、$s = 2$ 时，其位置编码的函数曲线与原始位置 $m = 8$ 的曲线接近（图 4.17）。

依据 《大语言模型：训练、精调与对齐》第 4.4.4 节（式 (4-70)）

PI 前后的曲线对比

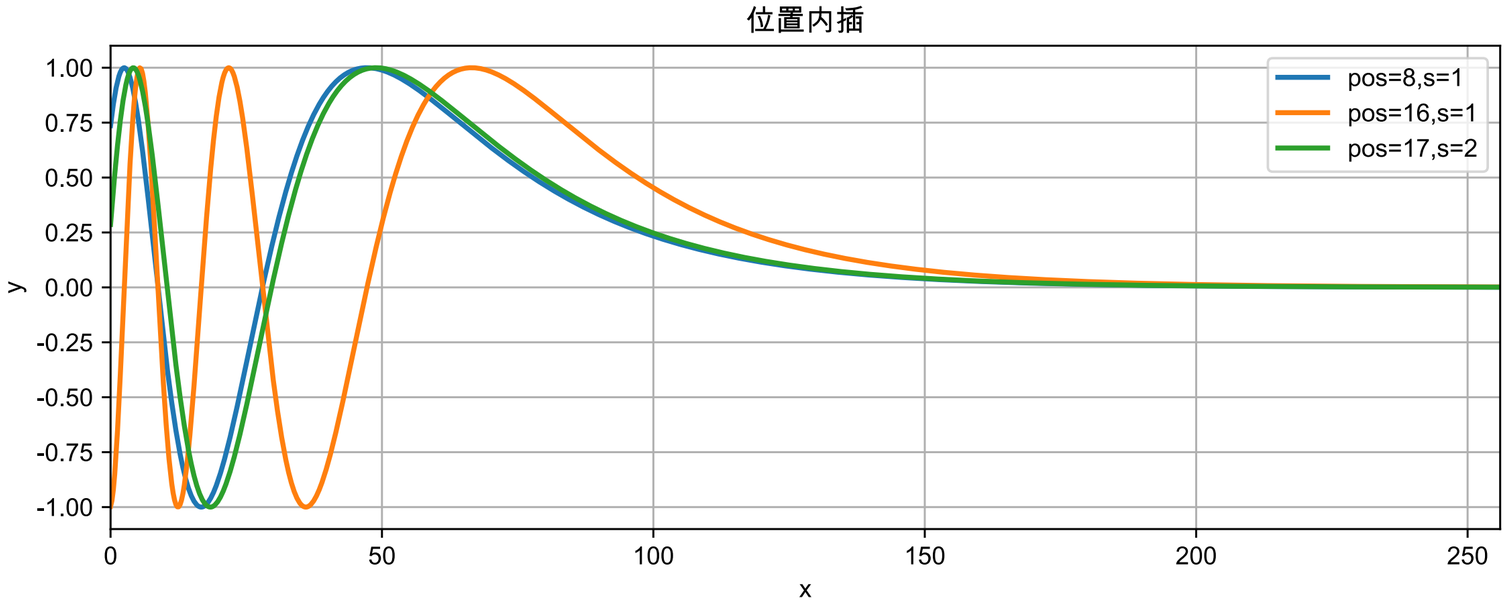


图 4.17　位置编码内插前后对比

- 原始位置 $m = 17$、内插系数 $s = 2$ 后，位置编码对应的函数曲线与原始位置 $m = 8$ 的曲线接近。
- 新位置落在模型预训练时见过的数值范围内，注意力分数不再异常。
- 代价：所有位置都被“挤”到一起，位置之间的间隔变小。

PI-RoPE 的特性

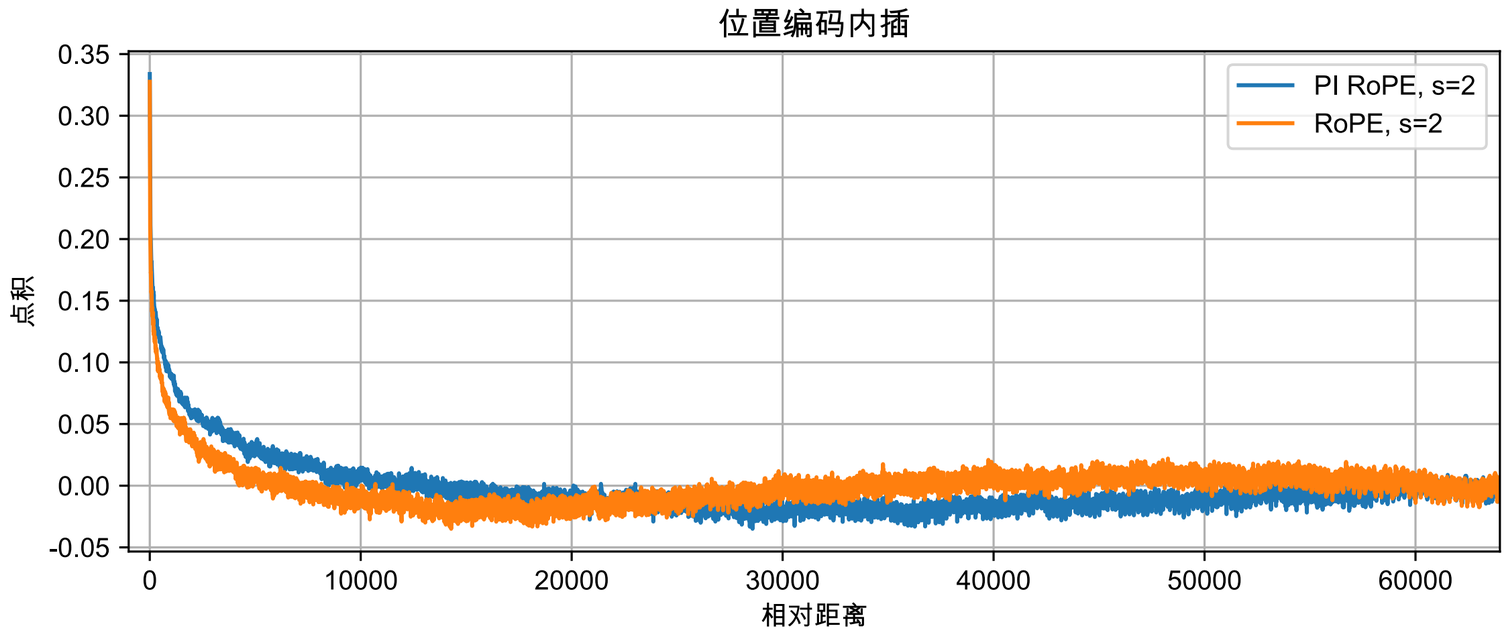


图 4.18　标准 RoPE 与 PI-RoPE 的注意力分数随相对位置衰减曲线对比图

- 近距（0–32k）：PI-RoPE 衰减比标准 RoPE 更慢，近距离的相对位置表达被削弱。
- 远距（32–64k）：PI-RoPE 衰减程度优于标准 RoPE，长距离信息保留更好。
- 短距离曲线更平缓，对应高频特征上的敏感性下降。

PI 损失近程精度的原因

PI 作用于周期性的位置编码函数 $\sin(\omega x)$ 时，相当于对所有频率分量做等比例压缩：

$$\omega \to \omega / s$$

内插系数 $s > 1$，每个频率分量都按同一比例降低——这是"等比例"的含义。

低维 = 高频

负责捕捉细微的位置变化。

频率被压缩后，对近距离位置的精细区分能力下降。

高维 = 低频

负责刻画较长程的位置关系。

变化相对较小，远距离感知得到增强。

净效果：远程能力提升，但近程精细位置信息受损，进而影响局部依赖关系的理解——这限制了 PI 扩展超长上下文时的最终效果，也促使 NTK-RoPE 采用非等比例的缩放。

NTK-RoPE

NTK 理论指出，高频分量易于学习但容易衰减或崩溃。NTK-RoPE 借鉴 PI 的思想，但改用非线性频率缩放：对高频分量（大维度索引、小旋转角）少缩放或不缩放，保留短距离的精细分辨力。

$$\left(\sin\left(\frac{m}{b'^{\frac{2k}{d}}}\right), \cos\left(\frac{m}{b'^{\frac{2k}{d}}}\right)\right) = \left(\sin\left(\frac{m}{(10\cdot b)^{\frac{2k}{d}}}\right), \cos\left(\frac{m}{(10\cdot b)^{\frac{2k}{d}}}\right)\right) \tag{4-71}$$

$$= \left(\sin\left(\frac{m}{10^{\frac{2k}{d}}\cdot b^{\frac{2k}{d}}}\right), \cos\left(\frac{m}{10^{\frac{2k}{d}}\cdot b^{\frac{2k}{d}}}\right)\right) \tag{4-72}$$

实现方式是用 $s>1$ 放大基数 $b'=sb$，这里以 $s=10$ 为例。把 $(10b)^{2k/d}$ 拆成 $10^{2k/d}\cdot b^{2k/d}$：缩放强度由 $10^{2k/d}$ 决定，随维度 k 变化。

- 低维度处（k 小）$10^{2k/d}$ 接近 1，旋转角几乎不变——高频分量被保留。
- 随维度增加，底数增大会使高维分量接近位置插值的形式：$m \to m/10^{2k/d} \approx m/10$（当 $k=2(d-1)$）。

NTK-RoPE 的基数调整

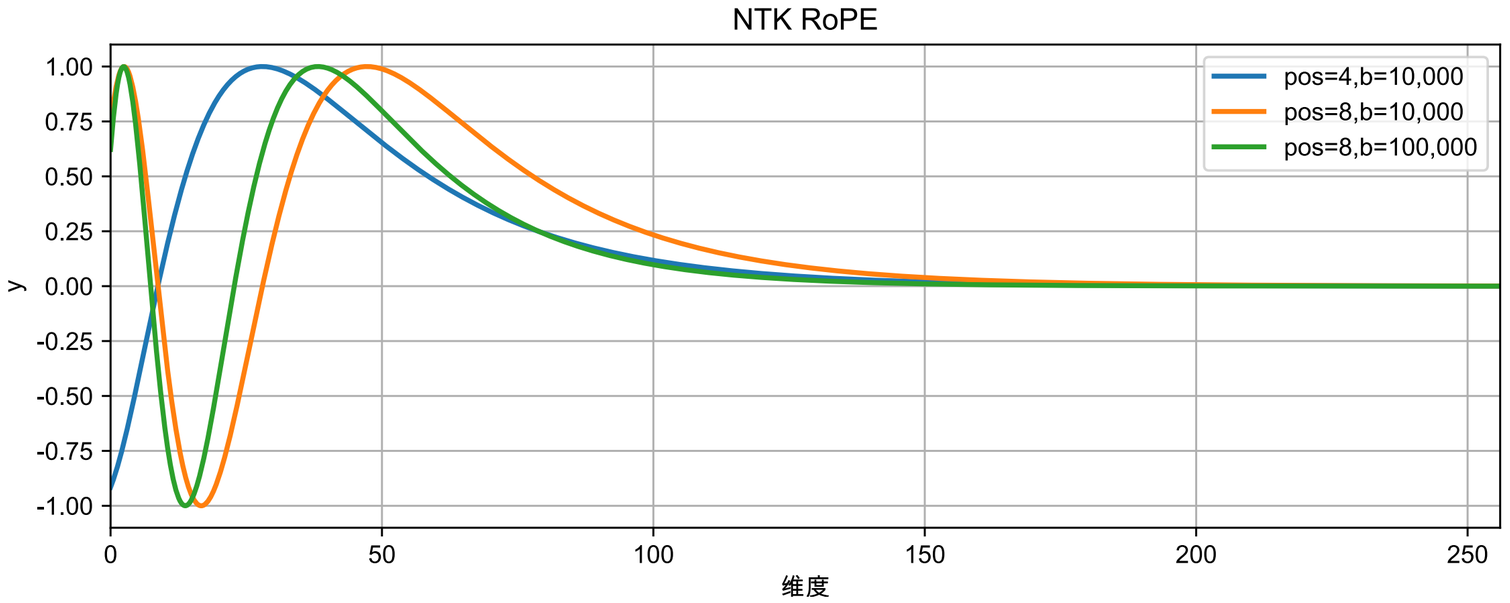


图 4.19　放大基数对位置编码向量影响

- 对比基数放大前后，高频分量的内插强度较小，低频分量的内插强度较大。
- 从左到右（维度由低到高），橙色线与绿色线的横向差距逐渐变大。
- 这正是“非线性缩放”：不是所有频率都被同等压缩。

64k 场景下的衰减特性

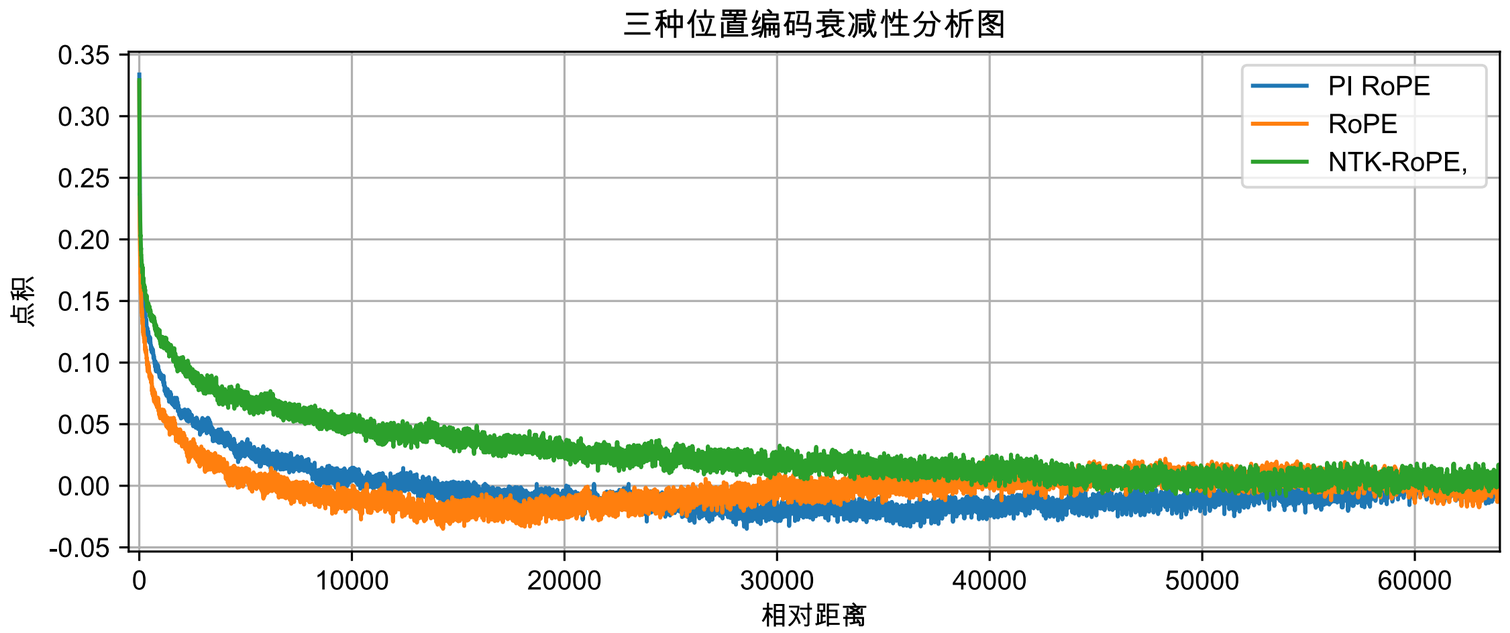


图 4.20　三种位置编码的衰减性可视化

- 测试结果表明，NTK-RoPE 的位置编码衰减特性稳定。
- 在 64k 的上下文长度场景中，衰减特性符合预期。
- 对比标准 RoPE 与 PI，NTK-RoPE 同时照顾了远程衰减与近程分辨。

两个频段的对比

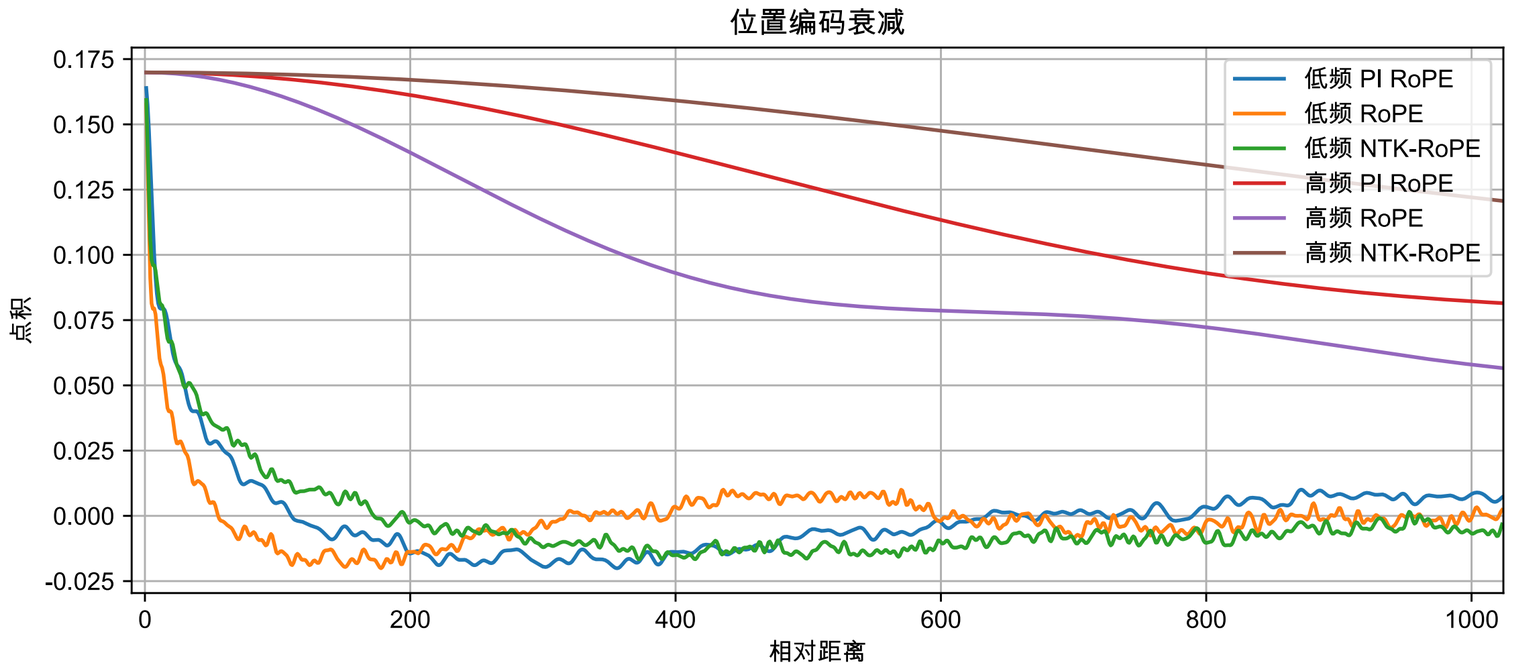


图 4.21　高低频位置编码的衰减性分析

- 为区分高频与低频分量的影响，分别对比 RoPE、PI、NTK-RoPE 在不同频段上的衰减。
- 结果显示，NTK-RoPE 在高频与低频区域都表现出符合理论预期的衰减行为。
- 相比 PI 的等比例压缩，NTK-RoPE 的频段表现更均衡。

长文本能力的评测方法

长文本扩展训练通常用 PPL 评估扩展效果；要考察模型在任意位置抓取信息的能力，还需专门的检索类基准。

指标 1

困惑度 PPL

在长文本语料上计算，衡量模型对新长度分布的适应程度。

指标 2

Needle in a Haystack

给定文本长度 L，把目标信息（“针”）放在相对位置 $y \in [0,1]$ 处，测量检索准确率。

怎么读热力图 对不同的 (L, y) 组合测量检索准确率，可构建二维热力图（横轴文本长度，纵轴相对位置）。若某些位置出现明显低谷，说明模型在该相对位置的“回忆”能力不足，与衰减曲线的行为相互印证。

4.5

小结

把 4.1–4.4 串起来：Llama 用四个模块替换做成一个纯解码器，KV-Cache 让推理可用，长文本继续预训练让上下文变长。

本节内容

结构要点回顾

与后续章节的衔接

Llama 结构要点

Llama 是一种自回归大语言模型，Llama-3 的最大参数规模达到 70B；其网络结构用多种手段替换了标准 Transformer 组件。

归一化

RMSNorm

只重缩放，计算省、梯度稳。

位置

RoPE

旋转注入相对位置，底数可调。

前馈

SwiGLU

门控激活增强特征表达。

注意力

GQA

减少投影参数与 KV-Cache 显存，注意力计算量不变。

- Llama-3 在多个基准上表现出色：8B 模型经 15T 词元训练后追平 Llama-2 70B。
- KV-Cache 预填充建缓存、解码只算新词元（4.3）；长文本靠继续预训练改造位置编码（4.4）。

依据 《大语言模型：训练、精调与对齐》第 4.5 节

Llama 的后续发展

生态

微调与多模态

Alpaca 对预训练模型做指令微调；LLaVA 将其扩展为多模态版本。

架构演进

MoE 替换前馈层

Mixtral、DeepSeek 系列用 MoE 替换标准前馈层，扩展参数规模并提升计算效率（详见 4.6）。

沿“如何把注意力做得更省”这条线，衍生出多种预训练模型；它们与 Llama 的纯解码器思路并列，各有取舍：

Linear Attention

线性注意力

Performer、Linformer：用核函数或低秩投影改写 Softmax 注意力，把复杂度从 $O(L^2)$ 降到 $O(L)$，代价是表达力弱于全注意力。

Sparse Attention

稀疏注意力

Longformer、BigBird：只保留局部窗口 + 少量全局词元，长文档任务上以更少的连接覆盖全序列。

Diffusion LLM

扩散式语言模型

LLaDA、Dream：用去噪扩散代替自回归逐词元生成，支持并行解码与双向上下文，采样步数与质量需权衡。

依据 《大语言模型：训练、精调与对齐》第 4.5 节

4.6

附录：稀疏混合专家系统

Llama-4 改用 MoE：用门控为每个词元只挑 Top-K 个专家，在扩大参数规模的同时控制计算量。

本节内容

稀疏 MoE 的计算

负载均衡损失

稀疏混合专家（MoE）

混合专家系统（MoE）是一种集成学习系统，包含门控与多个参数独立的专家；门控给出各专家对当前预测的重要性。

稠密 MoE

每个专家都参与计算。

算力消耗过多，失去了稀疏激活带来的效率优势。

稀疏 MoE

门控只取 Top-K 个专家参与计算。

计算量随 K 而非 N 增长，可在大参数规模下保持推理效率。

专家 在大语言模型中，一个前馈层或一个 SwiGLU 都可以作为一个专家 $E_i(\cdot)$。设共有 N 个专家，门控用线性层 $\boldsymbol{W}_{\text{gate}} \in \mathbb{R}^{d \times N}$ 实现。

稀疏 MoE 的计算步骤

$$\boldsymbol{s}_t = \boldsymbol{x}_t \boldsymbol{W}_{\text{gate}} \tag{4-73}$$

$$s'_{t,i} = \begin{cases} s_{t,i}, & s_{t,i} \in \texttt{Topk}(\{s_{t,j} \mid 1 \leqslant j \leqslant N\}, K), \\ -\infty, & \text{otherwise} \end{cases} \tag{4-74}$$

$$\boldsymbol{g}_t = \texttt{Softmax}(\boldsymbol{s}'_t) \tag{4-75}$$

$$\boldsymbol{y}_t = \sum_{i=1}^{N} g_{t,i} E_i(\boldsymbol{x}_t) \tag{4-76}$$

$\boldsymbol{x}_t \in \mathbb{R}^d$ 为第 t 个词元的特征，$\boldsymbol{s}_t \in \mathbb{R}^N$ 为各专家分数；未入选专家的分数置为 $-\infty$，Softmax 后其权重为 0；$E_i(\boldsymbol{x}_t) \in \mathbb{R}^d$。

- 式 (4-74) 的 $-\infty$ 与注意力掩码同理：Softmax 后权重恰为 0，未选中的专家不贡献输出。
- 式 (4-76) 只对被选中的 K 项有非零权重，故实际只前向 K 个专家。
- 一个序列的多个词元会按门控结果派发到不同专家：门控逐词元分配，每个词元激活自己的 Top-K。

依据 《大语言模型：训练、精调与对齐》第 4.6 节（式 (4-73)–(4-76)）

负载均衡损失

如果某些专家在初始阶段就被频繁选中，训练会一直偏向它们，其余专家始终得不到激活，自适应分配就失效了。为此引入负载均衡损失。

$$f(i) = \frac{1}{T} \sum_{t=1}^{T} \mathbb{I}\left(s_{t,i} \in \texttt{Topk}\left(\{s_{t,j} \mid 1 \leqslant j \leqslant N\}, K\right)\right) \tag{4-77}$$

$$g(i) = \frac{1}{T} \sum_{t=1}^{T} g_{t,i} \tag{4-78}$$

$$\mathcal{L}_{\text{load}} = \alpha \sum_{i=1}^{N} f(i)\, g(i) \tag{4-79}$$

$f(i)$：词元被分配到专家 i 的比率；$g(i)$：专家 i 的门控权重均值；α 为损失权重。

- 两项相乘再求和：某专家被选得越多、权重越大，惩罚越大，从而把词元推向冷门专家。
- 对于包含 MoE 网络的大型模型，还可以通过专家并行（Expert Parallel）提高并行性（见第 9 章）。

依据 《大语言模型：训练、精调与对齐》第 4.6 节（式 (4-77)–(4-79)）

思考题与实践题

1 LIama-3 有什么样的缩放定律结论？结合 15T 词元与 8B 追平 70B 讨论。

2 写出式 (4-2)–(4-7)，说明两处残差连接的作用。

3 从系统角度，推理时预填充和解码阶段 KV-Cache 各有什么特点？

4 GQA 中 $h=32$、$g=8$ 时，$\boldsymbol{W}_k$、$\boldsymbol{W}_v$ 的维度是多少？KV-Cache 相对 MHA 降为几分之几？

5 相较绝对位置编码，RoPE 为什么更优？从式 (4-28)–(4-31) 的四个交叉项出发说明。

6 写出无 KV-Cache 与有 KV-Cache 时生成第 $t+1$ 个词元的注意力计算，并比较计算量。

7 GQA 多头切分、RoPE 变换、KV-Cache 存储的顺序是什么？说清每一步作用在哪个张量上。

8 如何实现免训练的长度外推？以 PI、NTK-RoPE 与 YaRN 为例比较做法与代价。

依据 《大语言模型：训练、精调与对齐》第 4 章